

Large Language Models and Retrieval-Augmented Generation

Computer Science Study Notes, Part 3 of 6

1. Large Language Models

A large language model (LLM) is a neural network trained on very large amounts of text to predict the next token in a sequence. Most modern LLMs are based on the transformer architecture, which was introduced in the 2017 paper titled Attention Is All You Need. Given a prompt, the model generates a continuation one token at a time.

LLMs have important limitations. Their knowledge is frozen at the time of training, so they do not know about recent events or about private documents. They can hallucinate, which means producing fluent but false statements with confidence. They also have a context window, a maximum number of tokens they can read in a single request.

2. What Retrieval-Augmented Generation Is

Retrieval-Augmented Generation (RAG) is a technique that improves LLM answers by first retrieving relevant passages from a document collection and adding them to the prompt. The model is then instructed to answer using only that supplied context. RAG lets an assistant use private or up-to-date documents without retraining the model, and it makes it possible to show citations that point to the source of each answer.

RAG is different from fine-tuning. Fine-tuning changes the weights of the model by training it further, which is expensive and makes updating knowledge slow. RAG leaves the model unchanged and simply changes the information placed in the prompt, so adding a new document only requires indexing it.

A RAG pipeline has these stages: load the documents, clean the text, split it into chunks, create an embedding for each chunk, store the embeddings in a vector database, retrieve the most similar chunks for a user question, build a prompt from the question and the retrieved chunks, and generate the answer with the LLM.

3. Chunking

Documents are split into chunks because an embedding represents one piece of text, and because only a few relevant passages should be sent to the LLM. Fixed-size chunking cuts the text every N characters with an overlap between

neighbouring chunks. A common starting point is 500 to 1000 characters per chunk with an overlap of 10 to 20 percent.

The size involves a trade-off. Chunks that are too small lose the surrounding context, so a retrieved chunk may not contain enough information to answer. Chunks that are too large mix several topics, which dilutes the embedding and can exceed the maximum input length of the embedding model. Overlap helps because an answer that sits on a boundary between two chunks is still fully contained in at least one of them. Section-based chunking splits on headings or paragraphs and often keeps ideas more intact.

4. Embeddings and Similarity Search

An embedding is a list of numbers, a dense vector, that represents the meaning of a text. Texts with similar meaning have vectors that point in similar directions. Similarity is usually measured with cosine similarity, which ranges from -1 to 1. Chroma reports cosine distance, which equals 1 minus the cosine similarity, so a smaller distance means a closer match.

The sentence-transformers library provides ready-made embedding models. The model `all-MiniLM-L6-v2` is small and fast, produces 384-dimensional vectors, and truncates input after 256 word pieces. It is designed for English. For Arabic or mixed-language documents, `paraphrase-multilingual-MiniLM-L12-v2` is a better choice: it also produces 384-dimensional vectors and supports more than 50 languages, including Arabic.

The same embedding model must be used both when indexing the documents and when embedding the user question at query time. Vectors from different models are not comparable.

5. Vector Databases

A vector database stores embeddings together with the original text and metadata such as the source file name and page number, and it finds the nearest vectors to a query quickly. Popular choices are Chroma and FAISS. Chroma can persist its data to a folder on disk, so a backend can load the prepared index at startup instead of rebuilding it. Vector databases usually rely on approximate nearest neighbour algorithms, such as HNSW, to stay fast on large collections.

The retrieval step takes the user question, embeds it, and returns the top-k most similar chunks. The value k is commonly between 3 and 5. A distance threshold can be added: if even the best chunk is too far from the question, the system should say it cannot find the answer instead of guessing.

6. Grounded Prompting and Evaluation

A grounded prompt tells the model to answer only from the numbered context blocks, to cite the block numbers such as [1] and [2], and to reply with a fixed sentence like "I could not find this in the provided documents" when the context does not contain the answer. A temperature of 0 makes generation more deterministic, which suits factual question answering.

To evaluate a RAG system, test it on a set of questions and check three things: whether the retrieved context was relevant, whether the answer was grounded in that context, and whether the answer was correct. Include a few questions that are outside the documents, to verify that the assistant refuses instead of answering from its own general knowledge. Typical failure cases are retrieving the wrong chunk, an answer split across two chunks, and a model that ignores the context.

Ollama is a tool for running LLMs locally. After installing it, download a model with `ollama pull llama3.2:3b`, list downloaded models with `ollama list`, and chat with one using `ollama run llama3.2:3b`. The Ollama server listens by default on port 11434.